

Name Bhavya rana

Sapid 590034689

Batch 15

Algorithm

1. Start.
2. Input the size n of the array.
3. Validate n so that $1 \leq n \leq \text{MAX}$.
4. Input n array elements.
5. Repeatedly display the menu:
 - **1. Bubble Sort and Binary Search**
 - **2. Insertion and Deletion**
 - **3. Right Rotation by k positions**
 - **4. Exit**
6. Read the user's choice.
7. Use **switch** to perform the selected operation.
8. For **Bubble Sort and Binary Search**:
 - Sort the array in ascending order using Bubble Sort.
 - Display the sorted array.
 - Input the element to search.
 - Apply Binary Search.
 - Display its position if found; otherwise display "Element not found."
9. For **Insertion and Deletion**:
 - Display a submenu for insertion or deletion.
 - For insertion, input the element and a valid position.
 - Shift elements to the right and insert the new element.
 - For deletion, input a valid position.
 - Delete the element and shift remaining elements to the left.
10. For **Right Rotation**:
 - Input k .
 - Reduce it using $k = k \% n$.
 - Rotate all array elements right by k positions.
11. For **Exit**, terminate the program.
12. For any invalid menu choice, display an error message.
13. Stop.

Test-Case Table

Test	Operation	Input	Expected Output
1	Bubble Sort	{ 5 , 2 , 8 , 1 , 4 }	{ 1 , 2 , 4 , 5 , 8 }
2	Binary Search	Search 4	Element found

3	Binary Search	Search 7	Element not found
4	Insertion	Insert 10 at position 3 in {1,2,3,4}	{1,2,10,3,4}
5	Deletion	Delete position 2 from {10,20,30,40}	{10,30,40}
6	Right Rotation	{1,2,3,4,5}, k=2	{4,5,1,2,3}
7	Boundary Rotation	{1,2,3,4,5}, k=5	{1,2,3,4,5}
8	Invalid Position	Insert/delete outside valid range	Error message
9	Invalid Choice	Menu choice 7	Invalid choice
10	Exit	Choice 4	Program terminates

C Program

```
#include <stdio.h>
```

```
#define MAX 100
```

```
void display(int a[], int n);
void bubbleSort(int a[], int n);
void binarySearch(int a[], int n);
void insertElement(int a[], int *n);
void deleteElement(int a[], int *n);
void rightRotate(int a[], int n);
```

```
int main()
{
    int a[MAX];
    int n, choice, subChoice, i;

    /* Input and validation of array size */
    do
    {
        printf("Enter the number of elements (1-%d): ", MAX);
        scanf("%d", &n);

        if (n <= 0 || n > MAX)
            printf("Invalid size. Please enter again.\n");

    } while (n <= 0 || n > MAX);

    printf("Enter %d array elements:\n", n);
```

```

for (i = 0; i < n; i++)
    scanf("%d", &a[i]);

do
{
    printf("\n===== HDOC DAY-30 =====\n");
    printf("1. Bubble Sort and Binary Search\n");
    printf("2. Insertion and Deletion\n");
    printf("3. Rotate array to the right by k
positions\n");
    printf("4. Exit\n");
    printf("===== \n");
    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            bubbleSort(a, n);

            printf("\nArray after Bubble Sort:\n");
            display(a, n);

            binarySearch(a, n);
            break;

        case 2:
            printf("\n--- Insertion / Deletion ---\n");
            printf("1. Insert an element\n");
            printf("2. Delete an element\n");
            printf("Enter your choice: ");
            scanf("%d", &subChoice);

            switch (subChoice)
            {
                case 1:
                    insertElement(a, &n);
                    break;

                case 2:
                    deleteElement(a, &n);
                    break;

                default:

```

```

        printf("Invalid choice.\n");
    }
    break;

    case 3:
        rightRotate(a, n);

        printf("Array after right rotation:\n");
        display(a, n);
        break;

    case 4:
        printf("Exiting program...\n");
        break;

    default:
        printf("Invalid menu choice. Please try
again.\n");
    }

    } while (choice != 4);

    return 0;
}

/* Display array */
void display(int a[], int n)
{
    int i;

    for (i = 0; i < n; i++)
        printf("%d ", a[i]);

    printf("\n");
}

/* Bubble Sort */
void bubbleSort(int a[], int n)
{
    int i, j, temp;

    for (i = 0; i < n - 1; i++)
    {

```

```

        for (j = 0; j < n - i - 1; j++)
        {
            if (a[j] > a[j + 1])
            {
                temp = a[j];
                a[j] = a[j + 1];
                a[j + 1] = temp;
            }
        }
    }
}

/* Binary Search */
void binarySearch(int a[], int n)
{
    int key;
    int low = 0;
    int high = n - 1;
    int mid;
    int found = -1;

    printf("Enter the element to search: ");
    scanf("%d", &key);

    while (low <= high)
    {
        mid = (low + high) / 2;

        if (a[mid] == key)
        {
            found = mid;
            break;
        }
        else if (key < a[mid])
        {
            high = mid - 1;
        }
        else
        {
            low = mid + 1;
        }
    }

    if (found != -1)

```

```

        printf("%d found at position %d.\n", key, found + 1);
    else
        printf("%d not found in the array.\n", key);
}

/* Insert element */
void insertElement(int a[], int *n)
{
    int value, position, i;

    if (*n == MAX)
    {
        printf("Array is full. Insertion is not possible.\n");
        return;
    }

    printf("Enter the element to insert: ");
    scanf("%d", &value);

    printf("Enter position (1 to %d): ", *n + 1);
    scanf("%d", &position);

    if (position < 1 || position > *n + 1)
    {
        printf("Invalid position.\n");
        return;
    }

    for (i = *n; i >= position; i--)
        a[i] = a[i - 1];

    a[position - 1] = value;

    (*n)++;

    printf("Element inserted successfully.\n");
    printf("Array after insertion:\n");
    display(a, *n);
}

```

```

/* Delete element */
void deleteElement(int a[], int *n)

```

```

{
    int position, deleted, i;

    if (*n == 0)
    {
        printf("Array is empty. Deletion is not possible.
\n");
        return;
    }

    printf("Enter position to delete (1 to %d): ", *n);
    scanf("%d", &position);

    if (position < 1 || position > *n)
    {
        printf("Invalid position.\n");
        return;
    }

    deleted = a[position - 1];

    for (i = position - 1; i < *n - 1; i++)
        a[i] = a[i + 1];

    (*n)--;

    printf("Deleted element: %d\n", deleted);
    printf("Array after deletion:\n");
    display(a, *n);
}

/* Right rotation */
void rightRotate(int a[], int n)
{
    int k, i, j, temp;

    if (n == 0)
    {
        printf("Array is empty.\n");
        return;
    }

    printf("Enter the number of positions to rotate (k): ");
    scanf("%d", &k);

```

```
if (k < 0)
{
    printf("Invalid value of k.\n");
    return;
}

k = k % n;

for (i = 0; i < k; i++)
{
    temp = a[n - 1];

    for (j = n - 1; j > 0; j--)
        a[j] = a[j - 1];

    a[0] = temp;
}
}
```

```

Last login: Wed Sep 23 20:12:12 on ttys000
bhavya@Bhavyas-MacBook-Air ~ % nano day30.cpp
bhavya@Bhavyas-MacBook-Air ~ % clang day30.cpp -o day30
bhavya@Bhavyas-MacBook-Air ~ % ./day30
Enter the number of elements (1-100): 4 5 1 2 3
Enter 4 array elements:

===== HDOC DAY-30 =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
=====
Enter your choice: 1

Array after Bubble Sort:
1 2 3 5
Enter the element to search: 4
4 not found in the array.

===== HDOC DAY-30 =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
=====
Enter your choice: 2

--- Insertion / Deletion ---
1. Insert an element
2. Delete an element
Enter your choice: 3
Invalid choice.

===== HDOC DAY-30 =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
=====
Enter your choice: 5
Invalid menu choice. Please try again.

===== HDOC DAY-30 =====
1. Bubble Sort and Binary Search
2. Insertion and Deletion
3. Rotate array to the right by k positions
4. Exit
=====
Enter your choice: 4
Exiting program...
bhavya@Bhavyas-MacBook-Air ~ % █

```